

# Lab0-Report

Name: Gao Ning

Github name: Jacky-Ning-yyqz

Student ID: 25303050027

Date: 2026-09-25

## 1<sup>st</sup>.Purpose of the experiment:

### 1. 理解版本控制的概念与价值

认识版本控制在单人开发与多人协作中解决的核心问题——记录文件变化、支持版本回溯、支撑并行开发，理解为什么"手动另存为 xxx\_v2\_final.py"不是可扩展的方案。

### 2. 掌握 Git 的环境配置

学会安装与配置 Git，理解配置的三个层级（系统级 / 用户级 / 项目级）及其优先级；掌握 SSH 密钥对的生成方法，并能在 GitHub 上正确配置公钥，通过 SSH 协议与远程仓库建立安全连接。

### 3. 掌握 Git 的基本操作流程

理解工作区、暂存区、本地仓库三者的关系，掌握 `git init`、`git add`、`git commit`、`git status`、`git log` 等核心命令，并能解释 Git 为何要设计"暂存—提交"两个步骤。

### 4. 掌握分支管理机制

能够创建、查看、切换和合并分支，理解 fast-forward 合并与非 fast-forward 合并的区别，掌握 `--no-ff` 合并的意义。

### 5. 理解并解决合并冲突

掌握冲突产生的原因（两个分支修改了同一文件的同一区域），能够识别 `<<<<<<<`、`=====`、`>>>>>>>` 冲突标记，并独立完成冲突的解决与提交。

### 6. 掌握 GitHub 远程仓库的使用

学会基于模板仓库创建个人仓库，掌握 `git clone`、`git push`、`git pull` 等远程协作命令，区分本地分支与远程跟踪分支。

### 7. 建立规范的版本控制习惯

通过阅读 Commit Message 规范与 Gitflow 分支控制模型，理解规范化提交信息与分支管理的工程价值，初步建立良好的版本控制实践意识。

## 2<sup>nd</sup>.Experimental configuration and environment:

硬件与宿主系统: Intel x86\_64 (AMD64)

Linux 运行环境: 虚拟化方案—WSL2 (Windows Subsystem for Linux 2), 版本 2.7.8.0

Linux 发行: Ubuntu 24.04.4 LTS

系统架构: x86\_64

内核版本: 6.18.33.1-microsoft-standard-WSL2

开发与版本控制工具

工具: Git 2.43.0

编译器: gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)

构建工具: GNU Make

代码编辑器: Visual Studio Code (配合 WSL 远程扩展)

SSH 密钥算法: Ed25519

远程代码托管平台: GitHub

实验仓库

模板仓库 <https://github.com/ICS-26Fall-FDU/GitLab>

个人仓库 <https://github.com/Jacky-Ning-yyqz/GitLab>

仓库可见性: Public

## 3<sup>rd</sup>:Experimental procedures

### 3.1 环境配置

本次实验按照课程要求在 WSL (Windows Subsystem for Linux) 的 Ubuntu 24.04 环境中完成。

1. 确认 Git 已安装: 执行 `git --version`, 输出 `git version 2.43.0`。

2. 配置全局用户信息, 使提交记录能正确归属:

```
git config --global user.name "Jacky-Ning-yyqz"
```

```
git config --global user.email "418348136@qq.com"
```

3. 生成 Ed25519 密钥对: `ssh-keygen -t ed25519 -C "418348136@qq.com"`

4. 将公钥 `~/.ssh/id_ed25519.pub` 的内容添加到 GitHub 的 SSH and GPG keys 中。

5. 验证连接: 执行 `ssh -T git@github.com`, 输出 `Hi Jacky-Ning-yyqz! You've successfully authenticated, but GitHub does not provide shell access.`

6. 基于课程模板仓库创建个人仓库, 并克隆到本地: `git clone git@github.com:Jacky-Ning-yyqz/GitLab.git`

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\41834> wsl
jackyning@jackyning:/mnt/c/Users/41834$ ls ~/.ssh
id_ed25519 id_ed25519.pub
jackyning@jackyning:/mnt/c/Users/41834$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIEFUF+CRRC/QesFfQFvtjDkXNhFfj94In9wEOJRjiIOW 418348136@qq.com
jackyning@jackyning:/mnt/c/Users/41834$ ssh -T git@github.com
The authenticity of host 'github.com (20.205.243.166)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
Hi Jacky-Ning-yyqz! You've successfully authenticated, but GitHub does not provide shell access.
jackyning@jackyning:/mnt/c/Users/41834$
```

### 3.2 完成 main.c 中的 TODO

1. 查看 main.c, 其中 TODO 要求打印一句自定义的话。

2. 编辑文件, 新增一行 `printf("ics NB!\n");`

3. 编译运行验证: 执行 `make && ./main`, 程序正确输出内容; 执行 `make clean` 清理编译产物。

4. 提交并推送

### 3.3 分支管理与冲突解决

1. 创建并切换到 feature 分支: `git switch -c feature`

2. 在 feature 分支上将 main.c 中 printf("ics NB!\n"); 一行修改为 printf("feature branch version\n");, 提交
3. 切回 main 分支: git switch main
4. 在 main 分支上将同一行修改为不同内容 printf("main branch version\n");, 提交
5. 执行合并: git merge feature, 出现冲突提示:

```
jackyning@jackyning:~/GitLab$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
jackyning@jackyning:~/GitLab$
```

```
C main.c
1  #include <stdio.h>
2
3  int main()
4  {
    采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
5  <<<<<<< HEAD (当前更改)
6      printf("main branch version\n");
7  =====
8      printf("feature branch version\n");
9  >>>>>>> feature (传入的更改)
10     // @TODO: print a sentence you want.
11     printf("Hello, world!\n");
12 }
```

6. 打开 main.c, 可以看到 Git 插入的冲突标记:

```
<<<<<<< HEAD
    printf("main branch version\n");
=====
    printf("feature branch version\n");
>>>>>>> feature
```

Git 无法判断应保留哪一份修改, 因此将决定权交给使用者。

7. 解决冲突: 删除 <<<<<<<、=====、>>>>>>> 三行标记, 保留双方的修改 (两行都保留)。

```
jackyning@jackyning:~/GitLab$ git commit -m "merge: resolve conflict between main and feature"
[main 8a5f4ac] merge: resolve conflict between main and feature
jackyning@jackyning:~/GitLab$
```

8. 验证标记已清理干净: grep -n "<<<\|==\|>>>" main.c, 无任何输出。

9. 编译验证: make && ./main, 程序正常输出三行内容。

10. 提交解决结果并推送:

```
git add .
```

```
git commit -m "merge: resolve conflict between main and feature"
```

```
git push
```

11. 查看分支图确认合并结果: git log --oneline --graph

## 4<sup>th</sup>: Question Answers:

### 4.1 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

我没有多人协同开发的经历，但有一段单人反复迭代的实践，让我同样深刻体会到了版本管理的必要性。

我曾在本机部署过一个轻量级语言大模型。这个过程需要反复修改多个文件——推理脚本、模型配置以及若干辅助工具文件；其中同一个文件往往要经历多轮迭代，比如为了让模型在有限显存下顺利运行，脚本中的参数需要一遍遍地调整尝试。

在没有版本控制的情况下，这带来了几个具体的麻烦：

(1) 无法回退。某次修改导致程序跑不通时，之前能用的版本已经被覆盖，只能凭记忆手动改回去，常常改不回来。

(2) 无法对比。明知“上一版是能跑的”，却说不清这次究竟改了哪几行才导致失败。

(3) 只能靠文件名硬撑。用 `xxx_v1.cpp`、`xxx_v2_final.cpp`、`xxx_v2_final.cpp` 区分版本，时间一长自己也分不清哪个才是真正可用的。

(4) 磁盘堆积。同一份文件留下大量副本，既占空间又容易误用，造成 C 盘 D 盘大量空间浪费。

这段经历让我意识到：版本控制并不只是“多人协作工具”，它首先是“单人开发的后悔药”。Git 恰好针对性地解决了上述问题——每次提交都留下一个带说明的历史快照，随时可以回到任意版本；分支让我可以放心尝试激进的改动，失败就切回主线；`git diff` 能精确显示改动内容。

因此，虽然我没有多人协作的经历，但我清楚 Git 要解决的是什么问题。正因为亲手踩过这些坑，才更能理解它的价值。

### 4.2 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

Git 设计“暂存—提交”两个步骤，核心目的是把「我改了哪些东西」和「我想提交哪些东西」这两件事分离开。如果没有暂存区，`git commit` 就只能把工作区的所有改动一次性打包，使用者将失去选择的余地。

具体而言，这样的设计带来四个好处：

(1) 精确控制提交内容。假设我为了修复 bug A 修改了 `main.c`，顺手改了 `README.md` 的错别字，调试时又在 `test.c` 里加了几行临时代码。若只想提交与 A 相关的改动，只需 `git add main.c` 即可。没有暂存区的话，就只能把三个不相关的改动混在同一个提交里，或者手动把文件改回去再改回来。

(2) 便于把大改动拆成逻辑清晰的多个提交。一次编码可能涉及多个互不相关的功能，借助暂存区可以分多次 `add + commit`，使每次提交都是一个独立、完整、可理解的逻辑单元。这在事后追溯时尤为重要：当需要定位“哪次提交引入了 bug”或用 `git revert` 回退某次改动时，提交粒度越清晰，操作就越精准；反之若所有改动挤在一个巨型提交中，就只能整体回退。

(3) 提供了提交前审视的机会。`git add` 之后可以用 `git diff --staged` 查看“即将被提交的内容”，这能拦截误提交密码、调试用的 `printf`、忘记删除的临时文件等事故。

(4) 支持部分暂存。`git add -p` 可以把一个文件内的改动拆成若干块逐个确认，从而只提交一个文件中的部分改动。这在没有暂存区的模型中是做不到的。

可以用寄快递来类比：工作区是杂乱的房间，暂存区是打包好的箱子，仓库则是已寄出的包裹。房间里可能有一堆东西，但这次只想寄其中三样——先把它们挑进箱子(`git add`)，

确认装对了 (`git diff --staged`), 再封箱寄出 (`git commit`)。暂存区就是这个箱子, 它让人在封箱前还留有反悔和调整的余地。

#### 4.3 `git branch` 和 `git branch -a` 的区别是什么? 查阅资料并回答。

`git branch` 只列出本地分支; `git branch -a` (`-a` 是 `--all` 的简写) 会在此基础上额外列出远程跟踪分支, 即 `remotes/` 开头的条目。

我在自己的仓库中进行了实测验证。执行 `git branch` 时, 输出只有两个本地分支:

```
feature
* main
```

执行 `git branch -a` 后, 额外出现了两条:

```
feature
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main
```

其中 `remotes/origin/main` 记录的是远程仓库 GitHub 上 `main` 分支的状态 (`origin` 是远程仓库的默认别名)。此外, `remotes/origin/HEAD -> origin/main` 是一个符号引用, 指向远程仓库的默认分支, 克隆仓库时 Git 依据它决定本地默认检出哪个分支。

一个值得注意的细节是: `remotes/origin/feature` 并未出现, 因为我从未把 `feature` 分支推送到 GitHub。这印证了 `remotes/` 下列出的并非本地分支的镜像, 而是远程仓库上此刻真实存在的分支状态。

## 5<sup>th</sup>. Reading Insights and Reflections:

### 5.1 《Commit Message 规范》内容概括

这篇文章指出, Commit message 不应随意书写, 社区已经形成了成熟的规范, 其中 Angular 规范使用最为广泛。

规范化的提交信息有三方面价值:

(1) 便于快速浏览历史——`git log --pretty=format:%s` 让每次提交占一行, 看行首即可了解目的;

(2) 便于按关键词过滤检索, 例如 `git log --grep feature` 只显示新增功能的提交;

(3) 能够自动生成 Change log, 发布新版本时无需手工整理更新日志。

规范要求提交信息采用 Header / Body / Footer 三段式结构。其中 Header 是必需项, 格式为 `<type>(<scope>): <subject>`:

`type` 只能取 `feat` (新功能)、`fix` (修补 bug)、`docs` (文档)、`style` (格式)、`refactor` (重构)、`test` (测试)、`chore` (构建或辅助工具变动) 七种植。文章特别指出, 只有 `feat` 和 `fix` 会必然进入 Change log。

`scope` 用于说明影响范围, 为可选项。

`subject` 要求不超过 50 个字符, 以动词开头、使用第一人称现在时 (用 `change` 而非 `changed`)、首字母小写且结尾不加句号。

`Body` 部分用于说明改动的动机以及与旧行为的对比; `Footer` 则用于标注不兼容变动 (以 `BREAKING CHANGE` 开头) 或关闭 issue (如 `Closes #234`)。文章还介绍了 `Commitizen`、`validate-commit-msg` 等配套工具, 前者用于交互式生成合规信息, 后者可在提交时自动校验格式。

## 5.2 《Gitflow 使用规范》内容概括

这篇文章提出了一套应对多人协作的分支管理策略，其目标是让不同分支各司其职，避免协作过程中的代码冲突和版本混乱。文章强调，各团队的分支规范“万变不离其宗，都有 Gitflow 的影子”。

Gitflow 定义了五种分支：

**master (永久)：**与线上环境严格一致，只用于存放发布，每次提交后立即打 tag。

**develop (永久)：**从 master 分离，作为团队日常开发的公共集成点。

**feature (临时)：**一个新功能对应一个分支，从 develop 创建，开发完成后合并回 develop 并删除。

**release (临时)：**测试阶段使用，从 develop 创建并更新版本号，负责修复测试环境中发现的 bug，完成后合并到 master 和 develop 并删除。

**hotfix (临时)：**用于紧急修复线上 bug，直接从 master 创建，修复完成后合并到 master 和 develop 并删除。

全文贯穿三条核心原则：

(1) “从哪里来，最后回到哪里去”——分支从哪个分支创建，完成后就合并回该分支。这是文章反复强调的原则，用以保证分支图清晰不乱。

(2) 合并时使用 `--no-ff` 参数——禁用快进合并，强制执行三方合并，从而保留分支曾经存在的历史痕迹。若使用快进合并，这段分支历史就会消失。

(3) 临时分支用完即删——避免“僵尸分支”堆积，保持分支列表干净。

文章最后还给出了版本号约定：格式为 0.0.0，首位表示重大版本、中位表示功能迭代、末位表示热修复；在 master 分支上打 tag，不带 v 前缀，只使用数字和点号。

## 5.3 谈谈你对“为什么要学习 Git”这个问题的理解

第一，Git 是个人开发的“后悔药”。我在本地部署轻量级语言模型时，最痛苦的就是改坏了无法回头——旧版本已被覆盖，只能凭记忆硬改。Git 用一次次提交构成完整的时间线，`git checkout` 可以回到任意历史版本，`git diff` 能精确指出改动了什么。这让我敢于大胆尝试，因为知道随时可以退回去。

第二，Git 是多人协作的基础设施。本实验中我遇到的合并冲突，正是协作场景的缩影：两个分支修改了同一处代码，Git 无法替人决定保留哪一份，于是把决定权交还给人。虽然我的冲突是人为制造的，但它真实再现了多人并行开发时必然遇到的问题。而从《Gitflow 使用规范》可以看到，工业界已经通过分支职责的划分和合并原则，把这套流程工程化了。

第三，Git 让项目拥有可追溯的历史档案。规范的提交信息、清晰的版本号、有条理的分支结构，共同构成项目的档案。将来定位问题、追溯 bug、生成发布日志，全都依赖这份档案。这不是可有可无的“锦上添花”，而是软件工程的基本要求。

第四，Git 是本课程和后续学习的基础工具。本学期的所有实验都通过 GitHub 仓库提交，Git 不是可选项而是必需项。更重要的是，作为计算机专业的学生，无论将来从事开发、科研还是运维，Git 都是绕不开的通用技能。尽早掌握它，收益会体现在之后的每一次项目中。